

Primero antes de nada, creamos la red interna de docker, a la que le vamos a llamar mongo-cluste-net. Después los contenedores que creemos, que serán mongo 1, 2 y 3

Se conectarán a esta red y podrán comunicarse entre sí.

```
docker network create mongo-cluster-net
```

Una vez ejecutado este comando, queremos comprobar que la red existe, por lo que haremos un ls de las network

```
docker network ls
```

Y aparece mongo-cluster-net entre otras que tengo en mi máquina.

Entonces ahora lo que queremos hacer es crear los contenedores de docker para mongo 1, 2 y 3, que se llaman así para después se puedan localizar entre sí, siendo esto importante para el sharding y la replicación. Además, cogemos la última imagen oficial de mongo con mongo:latest

```
docker run -d --name mongo1 --network mongo-cluster-net mongo:latest tail -f /dev/null
```

```
docker run -d --name mongo2 --network mongo-cluster-net mongo:latest tail -f /dev/null
```

```
docker run -d --name mongo3 --network mongo-cluster-net mongo:latest tail -f /dev/null
```

Ponemos tail-f -f/dev/null, porque no queremos que se ejecute MongoDB todavía, de tal manera que dejamos hechos los contenedores pero sin arrancar mongo. Después arrancaremos los procesos mongod y mongos manualmente.

Una vez más, hacemos una comprobación de que todo salió correcto:

```
docker ps
```

~ took 18s

➤ docker ps

CONTAINER ID	IMAGE	COMMAND	CREATED
STATUS	PORTS	NAMES	
c84ceba8195	mongo:latest	"docker-entrypoint.s..."	About a minute ago
Up About a minute	27017/tcp	mongo3	
a8286043ff2c	mongo:latest	"docker-entrypoint.s..."	About a minute ago
Up About a minute	27017/tcp	mongo2	Up
4724b4aef5df	mongo:latest	"docker-entrypoint.s..."	About a minute ago
Up About a minute	27017/tcp	mongo1	Up
01e8a5ace47d	postgres:xine-grei	"/bin/sh -c 'service..."	3 weeks ago
Up 2 hours	0.0.0.0:35432->5432/tcp	xine-grei-pgsql	

Vemos aquí las imágenes de mongo1, mongo2 y mongo3, por lo que está correcto.

Vamos ahora entonces con Mongo 1.

Tendremos que crear los directorios dentro del contenedor, por lo que primero ejecutamos el siguiente comando:

```
docker exec -it mongo1 bash
```

Para poder entrar en una terminal dentro del mismo.

Una vez dentro de la terminal del contenedor, crearemos los directorios con estos comandos

```
mkdir -p /data/configdb  
mkdir -p /data/shard1  
mkdir -p /data/shard2  
mkdir -p /data/shard3  
mkdir -p /data/logs
```

Cada proceso de mongod, tendrá su propio directorio para evitarnos los conflictos

Ejecutamos la comprobación, y vemos que todo salió correcto:

```
root@4724b4aef5df:/# ls /data  
configdb db logs shard1 shard2 shard3
```

Ahora salimos del contenedor con:

Exit

Y repetimos el proceso en mongo2 y mongo3. Me ahorro la explicación, porque los comandos son los mismos.

```
> docker exec -it mongo1 bash  
root@4724b4aef5df:/# mkdir -p /data/configdb  
mkdir -p /data/shard1  
mkdir -p /data/shard2  
mkdir -p /data/shard3  
mkdir -p /data/logs  
root@4724b4aef5df:/# ls /data  
configdb db logs shard1 shard2 shard3  
root@4724b4aef5df:/# exit  
exit
```

~ took 2m51s

```
> docker exec -it mongo2 bash
```

```
root@a8286043ff2c:/# mkdir -p /data/configdb
mkdir -p /data/shard1
mkdir -p /data/shard2
mkdir -p /data/shard3
mkdir -p /data/logs
root@a8286043ff2c:/# ls /data
configdb db logs shard1 shard2 shard3
root@a8286043ff2c:/# exit
exit
```

```
~ took 26s
> docker exec -it mongo3 bash
root@c84ceba8195:/# mkdir -p /data/configdb
mkdir -p /data/shard1
mkdir -p /data/shard2
mkdir -p /data/shard3
mkdir -p /data/logs
root@c84ceba8195:/# ls /data
configdb db logs shard1 shard2 shard3
root@c84ceba8195:/# exit
exit
```

Con esto, ya tendremos las tres máquinas preparadas. Y el paso 1 estaría hecho

— — — — —

Ahora vamos a arrancar el replica set con rsConfServer.

Para ello, arrancamos el proceso mongod del config server en cada contenedor con el siguiente comando:

```
docker exec mongo1 mongod --configsvr --replSet rsConfServer --port 27019 --
dbpath /data/configdb --bind_ip_all --fork --logpath /data/logs/configsvr.log
docker exec mongo2 mongod --configsvr --replSet rsConfServer --port 27019 --
dbpath /data/configdb --bind_ip_all --fork --logpath /data/logs/configsvr.log
docker exec mongo3 mongod --configsvr --replSet rsConfServer --port 27019 --
dbpath /data/configdb --bind_ip_all --fork --logpath /data/logs/configsvr.log
```

Entonces arrancamos el replica set para usarlo como servidor de configuración.

Pero falta la parte importante: configurarlo con rs.initiate()

Para ello hacemos lo siguiente:

Nos conectamos desde mongo1 al proceso que acabamos de arrancar:

```
docker exec -it mongo1 mongosh --port 27019
```

Y una vez dentro de mongosh, ejecutamos lo siguiente:

```
rs.initiate({
  _id: "rsConfServer",
  configsvr: true,
  members: [
    { _id: 0, host: "mongo1:27019" },
    { _id: 1, host: "mongo2:27019" },
    { _id: 2, host: "mongo3:27019" }
  ]
})
```

Este es el paso que nos indica el enunciado cuando dice que recordemos lanzar el proceso, pero que no puede tener los parámetros por defecto, pues el replica set tiene más de un miembro. Ahora para comprobar el estado, y ver cual es el primary y el secondary, tenemos lo siguiente:

```
rs.status()
```

Teniendo esto como salida:

```
members: [
  {
    _id: 0,
    name: 'mongo1:27019',
    health: 1,
    state: 1,
    stateStr: 'PRIMARY',

    [...]

  {
    _id: 1,
    name: 'mongo2:27019',
    health: 1,
    state: 2,
    stateStr: 'SECONDARY',

    [...]

  {
    _id: 2,
    name: 'mongo3:27019',
    health: 1,
    state: 2,
    stateStr: 'SECONDARY',
```

Esto nos permite ver que el replica set está funcionando correctamente, siendo mongo1 el PRIMARY lo que significa que es el que coordina el replicar set y acepta escrituras, y los otros dos los SECONDARY que replican los datos del PRIMARY, y pueden convertirse en él si el actual falla. Vemos también que los nodos están activos con health:1.

—

Ahora vamos con el shard1, que lo arrancaremos con rsShard1

Para ello ejecutamos los siguientes comandos:

```
docker exec mongo1 mongod --shardsvr --replSet rsShard1 --port 27101 --dbpath /
data/shard1 --bind_ip_all --fork --logpath /data/logs/shard1.log
docker exec mongo2 mongod --shardsvr --replSet rsShard1 --port 27101 --dbpath /
data/shard1 --bind_ip_all --fork --logpath /data/logs/shard1.log
docker exec mongo3 mongod --shardsvr --replSet rsShard1 --port 27101 --dbpath /
data/shard1 --bind_ip_all --fork --logpath /data/logs/shard1.log
```

Lo que estamos haciendo aquí es arrancar tres procesos mongod que formarán el replica set rsShard1. Usamos el puerto 27101 en este caso y el directorio que creamos antes para guardar los datos.

Ahora de nuevo, nos conectamos al nodo de mongo1:

```
docker exec -it mongo1 mongosh --port 27101
```

Y ejecutamos, dentro de mongosh lo siguiente:

```
rs.initiate({
  _id: "rsShard1",
  members: [
    { _id: 0, host: "mongo1:27101" },
    { _id: 1, host: "mongo2:27101" },
    { _id: 2, host: "mongo3:27101" }
  ]
})
```

Y de nuevo, como en el paso anterior lo verificamos con status:

```
rs.status()
```

Obteniendo la siguiente salida:

```
members: [
  {
    _id: 0,
    name: 'mongo1:27101',
```

```

    health: 1,
    state: 1,
    stateStr: 'PRIMARY',

  {
    _id: 1,
    name: 'mongo2:27101',
    health: 1,
    state: 2,
    stateStr: 'SECONDARY',

  {
    _id: 2,
    name: 'mongo3:27101',
    health: 1,
    state: 2,
    stateStr: 'SECONDARY',

```

Igual que antes, vemos que mongo1 es PRIMARY y los otros dos SECONDARY, por lo que el rsShard1 está bien configurado.

—

Ahora vamos con el shard2 y 3, que serán exactamente los mismos pasos, pero con puertos diferentes, por lo que dejaré los comandos de arranque de los procesos, pero no explicaré el resto de pasos porque son básicamente los mismos.

Shard2:

```

docker exec mongo1 mongod --shardsvr --replSet rsShard2 --port 27201 --dbpath /
data/shard2 --bind_ip_all --fork --logpath /data/logs/shard2.log
docker exec mongo2 mongod --shardsvr --replSet rsShard2 --port 27201 --dbpath /
data/shard2 --bind_ip_all --fork --logpath /data/logs/shard2.log
docker exec mongo3 mongod --shardsvr --replSet rsShard2 --port 27201 --dbpath /
data/shard2 --bind_ip_all --fork --logpath /data/logs/shard2.log

```

Vuelve a ser el PRIMARY mongo1 y SECONDARY los otros dos.

Shard3:

```

docker exec mongo1 mongod --shardsvr --replSet rsShard3 --port 27301 --dbpath /
data/shard3 --bind_ip_all --fork --logpath /data/logs/shard3.log
docker exec mongo2 mongod --shardsvr --replSet rsShard3 --port 27301 --dbpath /
data/shard3 --bind_ip_all --fork --logpath /data/logs/shard3.log
docker exec mongo3 mongod --shardsvr --replSet rsShard3 --port 27301 --dbpath /
data/shard3 --bind_ip_all --fork --logpath /data/logs/shard3.log

```

Vuelve a ser el PRIMARY mongo1 y SECONDARY los otros dos.

Vale ahora que tenemos creados los replica sets de configuración y los que serán los shards, hay que formar un clúster “shardeado” que sea accesible. Para ello arrancamos mongos, que es el router del clúster.

Mongos no guarda datos, si no que recibe consultas y las redirige al shard correcto.

Entonces arrancamos mongos en mongo1:

```
docker exec mongo1 mongos --configdb rsConfServer/  
mongo1:27019,mongo2:27019,mongo3:27019 --bind_ip_all --port 27017 --fork --  
logpath /data/logs/mongos.log
```

Con este comando arrancamos el router del cluster en mongo1. La opción `—configdb` le dice dónde está el replica set de configuración, y `—port` es el puerto donde escuchará el router.

Ahora ejecutamos estos comandos:

```
docker exec mongo2 mongos --configdb rsConfServer/  
mongo1:27019,mongo2:27019,mongo3:27019 --bind_ip_all --port 27017 --fork --  
logpath /data/logs/mongos.log  
docker exec mongo3 mongos --configdb rsConfServer/  
mongo1:27019,mongo2:27019,mongo3:27019 --bind_ip_all --port 27017 --fork --  
logpath /data/logs/mongos.log
```

Para que como pide el enunciado, cada contenedor tenga su propio router mongos.

Ahora nos conectamos al router del cluster con el siguiente comando:

```
docker exec -it mongo1 mongosh --port 27017
```

Y ejecutamos dentro de mongosh, el comando que nos dice el enunciado:

```
sh.addShard("rsShard1/mongo1:27101,mongo2:27101,mongo3:27101")  
sh.addShard("rsShard2/mongo1:27201,mongo2:27201,mongo3:27201")  
sh.addShard("rsShard3/mongo1:27301,mongo2:27301,mongo3:27301")
```

Y ahora verificamos que todo esté correcto con

```
sh.status()
```

Y nos da la siguiente salida:

```
shards  
[  
  {  
    _id: 'rsShard1',
```

```

    host: 'rsShard1/mongo1:27101,mongo2:27101,mongo3:27101',
    state: 1,
    topologyTime: Timestamp({ t: 1773232025, i: 29 }),
    replSetConfigVersion: Long('-1')
  },
  {
    _id: 'rsShard2',
    host: 'rsShard2/mongo1:27201,mongo2:27201,mongo3:27201',
    state: 1,
    topologyTime: Timestamp({ t: 1773232025, i: 86 }),
    replSetConfigVersion: Long('-1')
  },
  {
    _id: 'rsShard3',
    host: 'rsShard3/mongo1:27301,mongo2:27301,mongo3:27301',
    state: 1,
    topologyTime: Timestamp({ t: 1773232025, i: 141 }),
    replSetConfigVersion: Long('-1')
  }
]

```

Aquí es importante destacar la parte del host, pues nos confirma que cada shard está compuesto por tres nodos, estando uno en cada contenedor. La conclusión que podemos sacar, es que la arquitectura del clúster ya está montada.

—

GESTIÓN DE DATOS

Una vez montada toda la arquitectura, vamos con las operaciones de gestión de datos.

Lo primero que debemos hacer es cargar los datos de las películas que tenemos en la base de datos “xine” (por lo menos en mi máquina).

